Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática
Engenharia e Gestão de Sistemas — 2025/2026

VoltGuard

IoT Energy Monitoring Platform

Grupo 4

Name	Student Number	Main Contribution
Diogo Silva	*108212*	*oam-service / API Gateway / Orchestration*
Francisco Murcela	*108815*	*notifications-service*
Gonçalo Lima	*108254*	*anomalies-service / Observability*
Martim Carvalho	*108749*	*composer backend/frontend / Authentication*

Live deployment: https://grupo4-egs-deti.ua.pt/
Source code: https://github.com/franciscomurcela/VoltGuard

June 4, 2026

Contents

1. Introduction

1.1 Problem Statement

Energy waste and undetected electrical faults are hidden costs in industrial and urban environments. Traditional monitoring approaches are reactive: operators are notified *after* a failure has occurred, or at best during a scheduled inspection. This results in:

- Unplanned downtime that disrupts operations;
- Energy overconsumption that goes unnoticed until invoicing;
- Manual inspection overhead that scales poorly with the number of monitored sites;
- No consolidated view across geographically distributed installations.

As IoT sensor hardware becomes commodity, the bottleneck shifts from *data collection* to *data interpretation at scale*. The challenge is not deploying sensors — it is knowing what the sensors are saying, in real time, across potentially hundreds of distributed nodes.

1.2 Solution: VoltGuard

VoltGuard is an end-to-end IoT energy monitoring platform that turns raw sensor telemetry into actionable intelligence. The core value proposition is built on three pillars:

> **Monitor** — Centralised, real-time ingestion of electrical measurements from all connected sensors in a single dashboard.
> **Detect** — Automatic anomaly detection using time-series forecasting (Prophet), with no manual threshold configuration.
> **Respond** — Immediate multi-channel alerting (email, SMS) to the right person the moment an anomaly is confirmed.

2. System Architecture

This chapter introduces the overall topology of VoltGuard: the services that compose it, the technologies behind each one, and how they communicate. Detailed responsibilities for each service are covered in Chapters 3–6.

2.1 Overview

VoltGuard follows a microservices architecture where each functional domain is isolated in its own deployable unit. All external traffic enters through a single ingress point: the Kong API Gateway, which then forwards each request to the appropriate upstream service. Service-to-service calls use internal Kubernetes DNS and never traverse the public network.

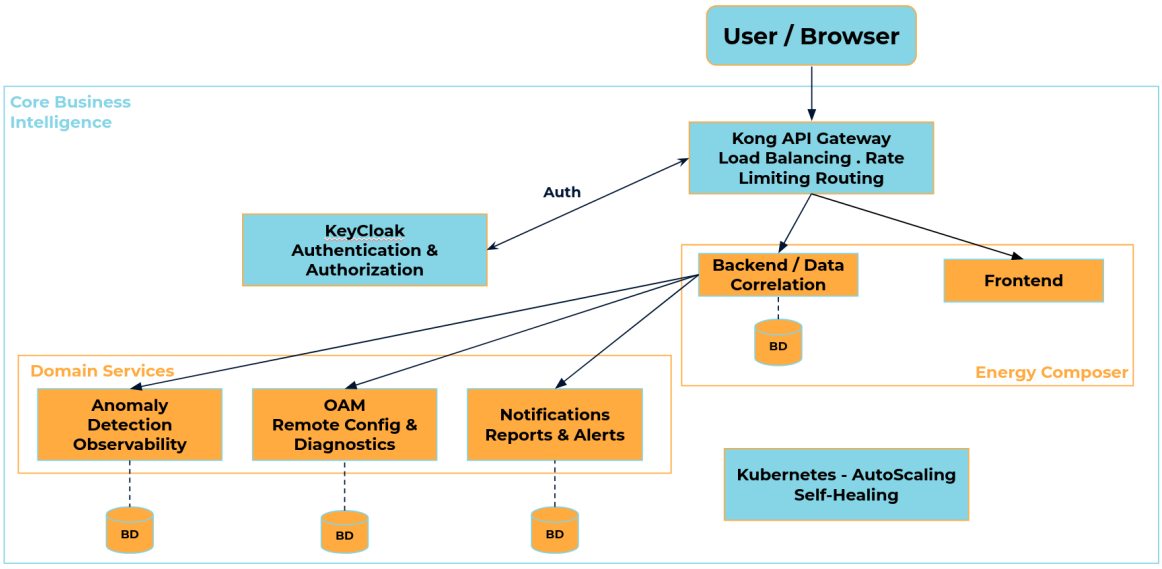

Figure 2.1: VoltGuard high-level architecture.

2.2 Technology Choices

Table 2.1: Technology stack per service

Component	Stack	Rationale
Composer Frontend	React + Vite	Component-driven SPA with fast HMR for development
Composer Backend	Node.js / Express	Lightweight SOA gateway; rich ecosystem for HTTP proxying
OAM Service	Rust / Axum / SQLx	Memory-safe, high-throughput service for the sensor inventory
Anomaly Detection	Python / FastAPI	Native Prophet bindings and async-friendly ML stack
Notifications	Python / Connexion	OpenAPI-first design enforced by the Connexion framework
API Gateway	Kong 3.7 (DB-less)	Declarative config, zero-DB overhead, native K8s support
Authentication	Keycloak 25	Production-proven OIDC implementation with PKCE support
Observability	Prometheus + Grafana	De-facto standard for cloud-native metrics collection and visualisation
Orchestration	Kubernetes	Self-healing pods, HPA, declarative infrastructure-as-code

2.3 Communication Patterns

- **Client → Kong**: Kong receives plain HTTP on port 8000.
- **Kong → Services**: Internal Kubernetes DNS.
- **Composer Backend → Peers**: Proxied REST calls with configurable timeout, retry count, and backoff.
- **Anomaly → Composer**: Outbound webhook `POST` to the URL declared in `VG_COMPOSER_WEBHOOK_URL` when an anomaly is confirmed; Composer then enqueues notifications.
- **Prometheus → Services**: Pull-based scrape every 15 seconds against each service's `/metrics` endpoint.
- **Authentication**: Unauthenticated requests are rejected with HTTP 401 by the Composer Backend; the frontend redirects to Keycloak, which issues a signed JWT. The backend validates every subsequent request locally against the Keycloak realm — Keycloak is never in the hot path for authenticated calls.
- **Sensor → alert pipeline**: A reading submitted to Anomaly Detection is evaluated against the Prophet confidence interval; if flagged, the service emits a webhook `POST` to the Composer Backend, which resolves the user's notification preferences and forwards a dispatch request to the Notifications service for email/SMS delivery.

3. Composer Service

3.1 Overview

The Composer Service is the user-facing core of VoltGuard. It consists of two sub-components: a **React frontend** that provides the operator dashboard, and a **Node.js backend** that acts as a Service-Oriented Architecture (SOA) gateway — aggregating and normalising data from all peer microservices before presenting it to the client.

> **Naming note.** This service is referred to as **Composer** throughout the report. The container images and Kubernetes service names retain the historical prefix `compositor-` (`compositor-frontend`, `compositor-backend`) for backward compatibility with the deployment pipeline.

3.2 Frontend

3.2.1 Technology & Build

The frontend is built with **React 18** and **Vite**, served as a static Single-Page Application from an nginx container. Authentication is handled client-side using **Keycloak-js** with the PKCE flow, which requires a secure context (HTTPS or localhost). The Keycloak URL is injected at build time via a Vite environment variable:

```
docker build \
  --build-arg VITE_AUTH_DISABLED=false \
  --build-arg
     VITE_KEYCLOAK_URL=https://grupo4-egs-deti.ua.pt/auth \
  -t
     registry.deti/grupo4-egs-deti.ua.pt/compositor-frontend:v7
     \
  composer-service/frontend/
```

Listing 3.1: Frontend Docker build arguments

3.2.2 Key Design Decisions

- Authentication variables are **build-time** (baked into the JS bundle), not runtime — this is a Vite constraint; `VITE_*` vars are resolved at `npm run build`. The implication is that switching environments requires rebuilding the image, not changing a Kubernetes env var.
- The nginx configuration sets `expires 1y; Cache-Control: public, immutable` for all static assets (JS, CSS, PNG), which maximises caching efficiency for content-hashed filenames.
- `VITE_AUTH_DISABLED` is independent of the backend's `AUTH_DISABLED` — both must be consistent or API calls will fail with HTTP 403, a misconfiguration we hit during development.

3.3 Backend

3.3.1 Technology & Role

The backend is a **Node.js / Express** API that acts as the single integration point for all data consumers. It owns no domain data directly; instead, it proxies and aggregates requests to OAM, Anomaly Detection, and Notifications.

3.3.2 Security

All endpoints are protected by Keycloak JWT validation. The backend verifies tokens against the Keycloak realm and rejects unauthenticated requests with HTTP 401. Sensitive configuration (client secrets, session keys, auth tokens) is loaded from Kubernetes Secrets at runtime, and is never embedded in plain text inside the deployment manifest or container image.

4. OAM Service

4.1 Overview

The **Operations, Administration and Maintenance (OAM)** service manages the lifecycle of physical sensors deployed in the field. It is responsible for sensor registration, firmware inventory, and keepalive tracking.

4.2 Technology

The OAM service is written in **Rust** using the **Axum** web framework, with **SQLx** for async PostgreSQL access. Rust was chosen for its memory-safety guarantees and predictable latency under load — important properties for a service that receives frequent keepalive heartbeats from a large sensor fleet.

4.3 Data Model

The service uses a **PostgreSQL 17** database managed by SQLx migrations. Persistent state includes:

- **Sensors** — unique identifier, district, firmware version, last-seen timestamp.
- **Firmware** — binary blobs stored on a persistent volume, with version metadata and checksum.
- **Keepalives** — timestamped heartbeat records used to detect sensor offline events.

4.4 API Surface

The OAM service exposes a REST API documented via OpenAPI (Swagger UI at `/swagger-ui`). Key endpoint groups:

Table 4.1: OAM service endpoint groups

Group	Path prefix	Description
Sensors	`/sensors`	CRUD for sensor registry
Firmware	`/firmware`	Upload, list, and assign firmware versions
Keepalive	`/keepalive`	Sensor heartbeat endpoint

4.5 Kubernetes Storage

Two PersistentVolumeClaims back the OAM service:

- `oam-postgres-data` — PostgreSQL data directory.
- `oam-firmware-storage` — Firmware binary blobs.

5. Anomaly Detection Service

5.1 Overview

The Anomaly Detection service is the intelligence layer of VoltGuard. It ingests sensor readings, models the expected baseline for each sensor, and flags readings that deviate significantly from that baseline.

5.2 Machine Learning Pipeline

The pipeline uses a robust time-series forecasting approach centred on Prophet:

- **Prophet** (Meta / Facebook) — a decomposable time-series forecasting model that captures trend and seasonality. The pipeline splits each incoming dataset using an **80/20 rule**: 80% of the historical data trains the model and establishes the baseline, while the remaining 20% is used for validation and **historical anomaly detection**. Any reading in the validation window that falls outside the generated confidence interval is flagged.

A reading is escalated to the Composer (and from there to the Notifications service) only if it falls outside the established Prophet confidence interval — a single, well-defined trigger that keeps the false-positive rate low without requiring per-sensor thresholds.

5.3 Predictive Forecasting

Beyond historical analysis, the persisted Prophet model unlocks proactive capabilities. The service can generate on-demand **forecasts** (`GET /forecasts/{sensor_id}`, see §5.5) to project future energy consumption. This allows facility managers to anticipate energy spikes and plan accordingly, shifting the platform from a purely reactive monitor toward a proactive intelligence tool.

5.4 Dynamic Calibration

VoltGuard exposes configurable parameters to system operators. Through the frontend dashboard, users can dynamically adjust the **Prophet uncertainty interval**: tightening the interval increases sensitivity and surfaces more anomalies, while widening it suppresses noise during expected transient events. The change takes effect for subsequent measurements without restarting the service.

5.5 API

The service is built with **FastAPI** and exposes:

- `POST /v1/measurements` — submit a batch of sensor readings for analysis. Triggers training, validation, and anomaly detection in a single request.
- `GET /v1/measurements` — retrieves historical sensor readings.

- `GET /v1/anomalies` — query confirmed anomaly events.
- `GET /v1/forecasts/{sensor_id}` — generate future consumption forecasts on-demand using the persisted model.
- `PUT /v1/models/{model_id}/config` — update the model sensitivity and configuration parameters.
- `GET /metrics` — Prometheus metrics endpoint.
- `GET /docs` — auto-generated FastAPI interactive documentation.

When an anomaly is confirmed, the service emits an outbound `POST` to the Composer webhook URL configured at deploy-time via `VG_COMPOSER_WEBHOOK_URL`; this is what triggers downstream notifications.

5.6 Secrets & Configuration

The service uses a `vault_loader.py` module to pull sensitive configuration (such as the `ANOMALY_APP_TOKEN`) from HashiCorp Vault at startup. In the Kubernetes deployment, Vault is replaced by Kubernetes Secrets injected as environment variables.

6. Notifications Service

6.1 Overview

The Notifications service is the multi-channel alert dispatch gateway. When the Composer receives an anomaly webhook from the Anomaly Detection service, it forwards a dispatch request to this service via HTTP `POST`. The Notifications service then evaluates user-defined routing profiles and executes delivery on each configured channel.

6.2 Technology Stack

The service is built on an OpenAPI-driven Python stack chosen for contract enforcement and clean separation between transport and channel logic:

- **Framework & API enforcement:** An **OpenAPI-first** design using **Connexion**, which generates the WSGI application directly from `openapi.yaml`.
- **Email dispatch:** Native **SMTP** using Python's standard `smtplib` and `email` packages, with TLS support and compatibility with modern relay providers (e.g., Gmail App Passwords).
- **SMS gateway:** The external **Twilio REST API** via the official `twilio` client library.
- **Extensible architecture:** The dispatch engine is decoupled from hardcoded credentials, reading provider parameters from the database. New channels can be registered or existing ones reconfigured at runtime without restarting the service.

6.3 Stored Data

The service uses a **MongoDB** database, providing loose coupling from the relational stores used by other services. Persistent state includes:

- **User Preferences** — unique user identifiers, cryptographic secrets, channel targets (phone numbers, emails), and alert dispatch policies (immediate vs. digest).
- **Notifications & Digest Queue** — operational logs of submitted payloads and a dedicated buffer for deferred alert aggregation.
- **Notification Audit** — a structured lifecycle event trail (`CREATED`, `DELIVERY_ATTEMPT`, `ABORTED_BY_PREFERENCE`, etc.) for diagnostics and compliance.

6.4 Supported Channels and API Features

All channel integrations are decoupled from the core business logic. Delivery is protected by an idempotency layer that prevents duplicate dispatch under network retries.

Table 6.1: Supported notification channels

Channel	Provider
Email	Native SMTP (`smtplib` with TLS)
SMS	Twilio SMS API

- **Idempotency key.** The `POST /v1/notifications` endpoint enforces an idempotency mechanism via the `Idempotency-Key` HTTP header, preventing duplicate notifications when callers retry after transient failures.
- **Dynamic preferences link.** On the first notification attempt for a given target, the system dynamically injects an opt-out URL signed with the user's unique cryptographic secret, enabling self-service profile management without an additional login step.

7. Kong API Gateway

7.1 Overview

Kong 3.7 operates in **DB-less** mode with a declarative configuration (`kong/kong.yml`). All routing rules, upstream targets, and plugins are defined in that single YAML file, making the gateway configuration fully reproducible and version-controlled.

7.2 Routing

Traffic arrives at the Kong proxy port and is routed to the appropriate upstream service based on the request path. Key routes:

Table 7.1: Kong routing table (Kubernetes deployment)

Path / Host pattern	Upstream service
`/`	Composer Frontend
`/api/*`	Composer Backend
`/auth/*`	Keycloak
`/oam/*`	OAM Service
`/notifications/*`	Notifications Service
`/anomaly/*`	Anomaly Detection

7.3 High Availability

Kong is one of the two components managed by a **HorizontalPodAutoscaler**. The HPA maintains at least 2 Kong replicas at all times, scaling up to 5 under CPU pressure. This ensures that no single Kong pod failure interrupts service (verified empirically in §8.5), and that gateway capacity grows proportionally with incoming traffic.

8. Deployment & Infrastructure

8.1 Kubernetes Overview

All production workloads run in the namespace `tenant-grupo4-egs-deti-ua-pt` on the DETI cluster. The deployment is fully declarative: applying `k8s/deployment.yaml` and `k8s/observability.yaml` brings up the entire platform from scratch.

8.1.1 Persistent Volumes

Table 8.1: PersistentVolumeClaims

Name	Consumer
`composer-data`	Composer Backend (session data)
`oam-postgres-data`	OAM PostgreSQL
`oam-firmware-storage`	OAM firmware binaries
`notifications-db-data`	Notifications Service
`keycloak-postgres-data`	Keycloak PostgreSQL

8.1.2 Secrets

All sensitive values (database passwords, API tokens, client secrets, session keys) are stored in a Kubernetes Secret resource applied from `k8s/secrets.yaml` (gitignored). Services consume them as environment variables injected by the kubelet: no secrets ever appear in a container image or deployment manifest.

8.2 Load Balancing

8.2.1 Kong as the Single Entry Point

The university Ingress Controller terminates TLS and forwards all traffic to the Kong Service. Kong then load-balances across its replica pods using the standard Kubernetes Service round-robin mechanism.

8.2.2 HorizontalPodAutoscaler

Two HPAs are configured, both CPU-triggered:

Table 8.2: HorizontalPodAutoscaler configuration

Component	Min replicas	Max replicas	Metric
Kong Gateway	2	5	CPU utilisation
Composer Frontend	2	5	CPU utilisation

The minimum of 2 replicas for both components ensures that a single pod failure never causes a service outage, so there is always at least one healthy pod handling requests while Kubernetes reschedules the failed one.

8.3 Observability

The observability stack is deployed via `k8s/observability.yaml` and consists of:

- **Prometheus** (`prom/prometheus:v2.53.1`) — scrapes all application services every 15 seconds.
- **Grafana** (`grafana/grafana:11.1.0`) — serves pre-provisioned dashboards at `/grafana`.

Table 8.3: Prometheus scrape targets

Job name	Metrics path
`anomaly-api`	`anomaly-api:8085/metrics`
`compositor-backend`	`compositor-backend:8080/metrics`
`notifications-service`	`notifications-service:8083/metrics`
`oam-service`	`oam-service:8080/metrics`

8.4 Stress Testing

Load testing was performed from a local machine using Apache Bench (`ab`) against the public HTTP endpoint:

```
ab -n 100000 -c 200 http://grupo4-egs-deti.ua.pt/
```

Listing 8.1: Stress test command

This generates 100 000 requests with a concurrency of 200. ApacheBench completed 99 978 out of 100 000 requests (Fig. 8.1) — a 99.978% success rate under sustained burst load. During the run:

- Both HPAs scaled to the 5-replica ceiling: Kong CPU peaked at 393% of target, the Composer Frontend at 359% (Fig. 8.2).
- Prometheus captured the CPU utilisation spike, and Grafana dashboards rendered the pod count growth and request rate in real time.
- Once the burst subsided, both HPAs stepped back down toward their minimum replica counts (Fig. 8.3).

Scaling events can be monitored live:

```
kubectl get events -n tenant-grupo4-egs-deti-ua-pt \
  --field-selector involvedObject.kind=HorizontalPodAutoscaler
    --watch
```

Listing 8.2: Monitor HPA scaling events

```
diogozeca@diogolap-asusg15:~$ ab -n 100000 -c 200 http://grupo4-egs-deti.ua.pt/
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking grupo4-egs-deti.ua.pt (be patient)
Completed 10000 requests
Completed 20000 requests
Completed 30000 requests
Completed 40000 requests
Completed 50000 requests
Completed 60000 requests
Completed 70000 requests
Completed 80000 requests
Completed 90000 requests
apr_socket_recv: Connection timed out (110)
Total of 99978 requests completed
```

Figure 8.1: ApacheBench executing 100 000 requests at concurrency 200 against the live cluster — 99 978 requests completed.

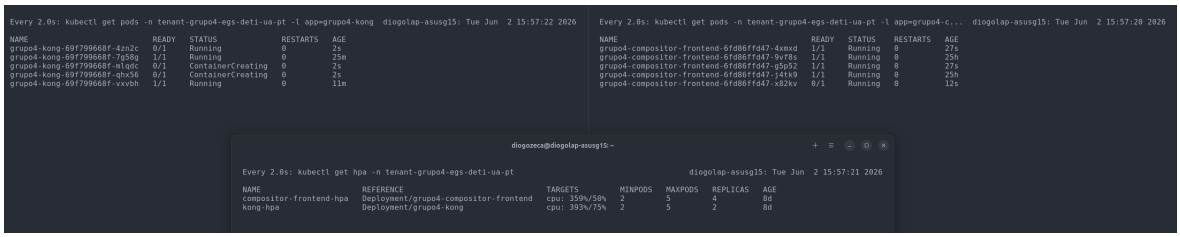

Figure 8.2: Kong CPU at 393% and Composer Frontend at 359% of target — new pods enter `ContainerCreating` as the HPA responds to the spike.

```
diogozeca@diogolap-asusg15:~$ kubectl get events -n tenant-grupo4-egs-deti-ua-pt --field-selector involvedObject.kind=HorizontalPodAutoscaler --watch
LAST SEEN   TYPE     REASON              OBJECT                                                  MESSAGE
13m         Normal   SuccessfulRescale   horizontalpodautoscaler/compositor-frontend-hpa   New size: 5; reason: cpu resource utilization (percentage of request) above target
11m         Normal   SuccessfulRescale   horizontalpodautoscaler/compositor-frontend-hpa   New size: 2; reason: All metrics below target
13m         Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 5; reason: cpu resource utilization (percentage of request) above target
10m         Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 4; reason: All metrics below target
9m50s       Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 3; reason: All metrics below target
8m35s       Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 2; reason: All metrics below target
0s          Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 3; reason: cpu resource utilization (percentage of request) above target
0s          Normal   SuccessfulRescale   horizontalpodautoscaler/compositor-frontend-hpa   New size: 5; reason: cpu resource utilization (percentage of request) above target
0s          Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 5; reason: cpu resource utilization (percentage of request) above target
0s          Normal   SuccessfulRescale   horizontalpodautoscaler/compositor-frontend-hpa   New size: 2; reason: All metrics below target
0s          Normal   SuccessfulRescale   horizontalpodautoscaler/kong-hpa                  New size: 4; reason: All metrics below target
```

Figure 8.3: HPA event stream: both HPAs scale up to 5 replicas under load, then step back down to 2–4 as CPU drops below target.

8.5 Resilience Testing

To validate that the platform tolerates pod failures during live traffic, a resilience test was performed with simultaneous pod termination under load. The full self-healing lifecycle is captured in Fig. 8.4.

Upon pod deletion:

1. The deleted pod transitions to `Terminating`.
2. Kubernetes immediately schedules a replacement pod, which enters `ContainerCreating` state.
3. The surviving replica continues serving requests.
4. The replacement pod reaches `Running` within approximately 10–20 seconds.
5. The HPA re-evaluates and restores the desired replica count.

The `ab` run completes with zero or negligible failed requests — any failures are limited to connections in-flight at the exact moment of termination — confirming that the platform is resilient to single-pod failures at the gateway layer.

(a) Initial state: 2 Frontend pods `Running`; `kubectl delete pod` command issued.

(b) Target pod transitions immediately to `Terminating`; second pod remains `Running`.

(c) Replacement pod reaches `Running` within ≈13 s — cluster restored to 2 healthy replicas.

Figure 8.4: Pod self-healing lifecycle: from deletion through `Terminating` to a fresh `Running` replacement.

9. Conclusions

VoltGuard successfully delivers a production-grade IoT energy monitoring platform within the constraints of a university project.

9.1 Key Achievements

- **End-to-end functionality** — from sensor data ingestion, through ML-based anomaly detection, to real-time multi-channel alerting.
- **Resilient deployment** — Kubernetes HPA with a minimum of 2 replicas for both the gateway and the frontend ensures the platform survives pod failures without downtime, empirically demonstrated in §8.5.
- **Observability** — Prometheus and Grafana provide full visibility into the platform's runtime behaviour, enabling proactive detection of performance degradation.
- **Security** — Keycloak OIDC with PKCE enforces authenticated access; secrets are never stored in version control or container images.
- **Developer experience** — declarative Kubernetes manifests, a local Docker Compose stack for development, and per-service Swagger UIs make the platform accessible to all team members.

9.2 Limitations

A balanced view of the platform must acknowledge what was *not* solved within the project window:

- **No scheduled Prophet retraining.** Models are trained on the dataset submitted with each request; long-term drift across weeks or seasons is not handled.
- **In-memory caches.** The Composer backend caches OAM responses for short TTLs (4 s for summary, 8 s for district stats) in process memory. Cache is lost on restart — acceptable for a 4-second window but not durable.
- **Single-region deployment.** The platform runs in one namespace on one cluster. Multi-region failover is out of scope.
- **Simulated sensor fleet.** Sensors are emulated through `scripts/bring-sensors-online.sh` and a JSON manifest rather than real hardware. The protocol surface is identical, but no real telemetry has been ingested.
- **Notification delivery best-effort.** If SMTP/Twilio is unreachable, failures are logged but not retried beyond the digest queue; a dedicated retry policy would harden production use.

9.3 Lessons Learned

Several non-obvious lessons emerged from running the project on a shared K3s cluster:

- **Build-time vs. runtime configuration is a real trap.** The frontend's Keycloak URL is baked into the JS bundle at `npm run build`; changing it requires a new image. We hit this when a stale image continued to point at `localhost:8081` after a deployment change.
- **Image rebuild discipline matters.** A bug fix that exists only in source code is not deployed. We had a district-name normalisation fix sitting in `metricsController.js` for several iterations before noticing the running pod still served the old code — the cluster has no CI that rebuilds on merge.
- **Idempotency keys pay for themselves.** The `Idempotency-Key` header on `POST /v1/notifications` prevented duplicate alerts during a Twilio rate-limit retry storm. Removing it for a quick test immediately produced double-sends.
- **Browser secure-context requirements bite quietly.** Keycloak-js's PKCE flow needs `crypto.subtle`, which browsers only expose over HTTPS or `localhost`. A plain-HTTP test deployment looked broken because the auth library silently degraded.
- **Diacritic-safe identifiers.** Mapping district names like "Setúbal" or "Bragança" to ASCII map polygon IDs required Unicode NFD normalisation. The naïve `toLowerCase().replace(/\s+/g,'_')` silently drops these districts from the dashboard.

9.4 Future Work

- Scheduled Prophet retraining with a configurable cadence per sensor.
- Persistent anomaly state in a dedicated time-series store (e.g. TimescaleDB or InfluxDB) so dashboards survive service restarts.
- Real-hardware integration trial with a small fleet of ESP32-class sensors over MQTT.
- End-to-end tracing with OpenTelemetry so a single anomaly can be followed from sensor through Composer, Anomaly, and Notifications.
- Cost-aware autoscaling that weights HPA decisions against energy or cloud cost, not just CPU.